TRAKYA ÜNİVERSİTESİ · BİLGİSAYAR MÜHENDİSLİĞİ

Dosya Organizasyonu

Hafta 3 · Dosya İşlemleri ve algoritmalar

Doç. Dr. Emir Öztürk

2025/2026 Güz Yarıyılı

ÇIKIŞ NOKTASI

Donanımdan yazılıma

Geçen hafta: erişim maliyetini belirleyen donanım

Bu hafta: bu donanımı kullanma biçimi

Dosya üzerindeki temel işlemler ve C++ karşılıkları

İsteğin işletim sistemi katmanlarında izlediği yol

Tamponlamanın başarımdaki yeri

Üçüncü haftanın çıkış noktası, ikinci haftada incelenen donanımın yazılım tarafından nasıl kullanıldığıdır. İkinci haftada erişim maliyetinin arama, dönme ve aktarım bileşenlerinden oluştuğu ve tasarımın hedefinin erişim sayısını azaltmak olduğu gösterilmiştir. Bu hafta, bir programın dosya üzerinde gerçekleştirdiği temel işlemlerden başlanarak bu işlemlerin işletim sistemi katmanlarında nasıl karşılandığı ele alınmaktadır. Haftanın sonunda verinin sayfa içinde nasıl yerleştirildiği incelenmekte ve kayıt biçimlerinin işleneceği dördüncü haftaya geçiş yapılmaktadır.

1

Temel dosya işlemleri

Açma, kapama, okuma, yazma, konumlandırma

Bu bölümde bir programın dosya üzerinde gerçekleştirdiği temel işlemler ele alınmaktadır. İşlemler açma, kapama, okuma, yazma ve konumlandırma başlıkları altında toplanmaktadır. Bölümün amacı, bu işlemlerin C++ karşılıklarını tanıtmak ve her birinin altında hangi sistem hizmetinin çalıştığını göstermektir.

KAVRAM

Fiziksel ve mantıksal dosya



Program hattın yalnızca kendi ucunu bilir. Mantıksal adı fiziksel dosyaya bağlamak işletim sisteminin işidir.

Fiziksel dosya: ortamda gerçekten duran bayt kümesi

Mantıksal dosya: programın gördüğü uç

Bir diskte binlerce fiziksel dosya bulunabilir

Bir program aynı anda yaklaşık yirmi dosya kullanır

4

Bir diskte ya da teypte dosya denildiğinde orada gerçekten duran bir bayt kümesi kastedilmektedir. Bu anlamıyla dosya fiziksel olarak vardır ve bir sürücüde binlercesi bulunabilir. Programın gördüğü dosya ise farklıdır: program yalnızca kendi ucunu bildiği bir hattan bayt alıp göndermekte, baytların nereden geldiğini ya da nereye gittiğini bilmemektedir. Bu hat mantıksal dosya olarak adlandırılmaktadır. Bağlantının kurulması işletim sisteminin işidir. Bir program aynı anda birden fazla mantıksal dosya kullanabilmekte, bu ad yalnızca programın içinde anlam taşımaktadır.

İŞLEM

Dosyanın açılması

Bayrak	Anlam
O_RDONLY · O_WRONLY · O_RDWR	okuma · yazma · okuma ve yazma
O_CREAT	dosya yoksa oluştur
O_TRUNC	varsa içeriğini sil
O_APPEND	her yazma dosyanın sonuna eklenir
O_EXCL	O_CREAT ile birlikte: dosya varsa hata döndür

İki seçenek: var olan dosyayı açmak, yeni dosya oluşturmak

Açma dosyanın içeriğini bozmaz, konum başa gelir

Dönen değer mantıksal tanıtıcıdır, hata durumunda negatiftir

Koruma kipi sekizli verilir: 0751 → sahip rwe, grup r-x, herkes --x

Açma işleminde iki seçenek bulunmaktadır: var olan bir dosyayı açmak ya da yeni bir dosya oluşturmak. Açma, dosyayı kullanıma hazır duruma getirmekte, içeriğini bozmamakta ve konumu dosyanın başına yerleştirmektedir. Yeni oluşturulan bir dosyanın içeriği bulunmadığından ilk anlamlı işlem yazmadır. İsteğin davranışı bayraklarla belirlenmektedir. Bayraklar bit düzeyinde birleştirilerek verilmektedir. Oluşturma isteniyorsa ayrıca koruma kipi belirtilmektedir. Bu kip, sahibin, grubun ve diğer kullanıcıların okuma, yazma ve çalıştırma yetkilerini sekizli düzende kodlamaktadır.

İŞLEM

Kapatma ve tampon boşaltma

Kapatmak, mantıksal ad ile fiziksel dosyanın bağını koparır

- Ad başka bir dosya için yeniden kullanılabilir

Veri diske bayt bayt değil blok blok taşınır

- Yazılanlar önce tamponda birikir

Kapatma, bekleyen tamponun boşaltıldığını güvence altına alır

Program normal sonlanırsa dosyalar örtük olarak kapatılır

Açık kapatma iki nedenle yazılır

- Kesinti durumunda veri kaybını önlemek
- Mantıksal adı yeniden kullanıma açmak

Kapatmada mantıksal ad serbest kalmakta ve başka bir dosya için yeniden kullanılabilmektedir. Çıktı için kullanılan bir dosyada kapatmanın ikinci ve daha önemli bir işlevi bulunmaktadır. Veri ikincil belleğe bayt bayt değil blok blok taşındığından, işletim sistemi yazılan baytları hemen göndermemekte, bir tamponda biriktirmektedir. Kapatma bu tamponun boşaltıldığını ve yazılan her şeyin dosyaya ulaştığını güvence altına almaktadır. Program normal sonlandığında dosyalar zaten kapatılmaktadır. Açıkça kapatma, kesinti durumunda veri kaybını önlemek ve mantıksal adı yeniden kullanıma açmak için yazılmaktadır.

İŞLEM

Okuma ve yazma

Okuma isteğinin üç bileşeni vardır

- Kaynak: açılmış mantıksal dosya adı
- Hedef: belleğin ilk adresi
- Uzunluk: bayt sayısı

Yazmada yön değişir, bileşenler aynı kalır

C akımları: stdio.h, düşük düzeyli ve açık

C++ akım sınıfları: iostream.h ve fstream.h

- Adresi derleyici bulur, tip aşırı yüklemeyle seçilir

Düşük düzeyli bir okuma isteği üç bilgi gerektirmektedir. Birincisi, verinin alınacağı mantıksal dosya adıdır. Bu ad kullanılmadan önce dosyanın açılmış olması zorunludur. İkincisi, okunan verinin yerleştirileceği bellek bloğunun ilk adresidir. Üçüncüsü, aktarılacak bayt sayısıdır. Yazma isteğinde yön değişmekte, bileşenler aynı kalmaktadır. C akımları bu üç bilgiyi açıkça istemekte ve işlemi sistem düzeyine yakın bir biçimde göstermektedir. C++ akım sınıfları ise adresi kendisi bulmakta ve aktarılacak tipi aşırı yükleme çözümlemesiyle seçmektedir.

İŞLEM

Dosya sonunun algılanması

İşletim sistemi konumu okuma/yazma göstericisiyle izler

C akımlarında okuma, okunan öğe sayısını döndürür

- Sıfır dönmesi dosya sonu demektir

C++ akım sınıflarında akımın durumu sorgulanır

- Önceki işlem başarısızsa fail doğru döner

Döngü, okumanın sonucuna göre kurulur

Sık hata: okumadan önce dosya sonunu sınamak

- Son kayıt iki kez işlenir ya da hiç işlenmez

Okuma döngüsünün doğru kurulması, dosya sonunun nasıl algılandığına bağlıdır. İşletim sistemi dosya içindeki konumu bir okuma yazma göstericisiyle izlemekte, her okumadan sonra göstericiyi ilerletmektedir. C akımlarında okuma işlevi okunan öğe sayısını döndürmekte, bu değerin sıfır olması dosya sonuna gelindiğini göstermektedir. C++ akım sınıflarında ise akımın durumu sorgulanmaktadır. Önceki işlem başarısız olmuşsa ilgili sorgu doğru dönmektedir. Döngünün okumanın sonucuna göre kurulması gerekmektedir. Okuma yapılmadan önce dosya sonunun sınandığı kalıplarda son kayıt ya iki kez işlenmekte ya da hiç işlenmemektedir.

İŞLEM

Arama ve konumlandırma

- Dosya, çok büyük bir bayt dizisi gibi görülür
- Arama, göstericiyi doğrudan istenen konuma taşır
  - 373. konuma gitmek 372 baytı okumayı gerektirmez
- Uzaklık bir başlangıç noktasına göre verilir
  - Dosyanın başı · geçerli konum · dosyanın sonu
- Uzaklık negatif olabilir, geniş tamsayı tipinde tutulur
- C++ akımlarında iki ayrı gösterici bulunur
  - Okuma ve yazma göstericileri birlikte hareket etmeyebilir

Sıralı okumada her bayt sırayla geçilmekte, gösterici kendiliğinden ilerlemektedir. Bazı durumlarda ise sırayla ilerlemeden doğrudan bir konuma gitmek gerekmektedir. Bu işlem arama olarak adlandırılmakta ve en az iki bilgi istemektedir: hangi dosyada arama yapılacağı ve baştan kaç konum ileri gidileceği. Üç yüz yetmiş üçüncü konuma gitmek, önceki üç yüz yetmiş iki konumu okumayı gerektirmemektedir. Uzaklık bir başlangıç noktasına göre verilmektedir. Dosyanın başı, geçerli konum ve dosyanın sonu kullanılabilmektedir. C++ akım sınıflarında okuma ve yazma için iki ayrı gösterici bulunmakta, bunların birlikte hareket edeceği garanti edilmemektedir.

BIÇIM

Yazım biçimleri ve özel karakterler

Metin kipinde sistem dosyaya davetsiz karakter ekleyebilir

- Dosya sonuna denetim karakteri eklenmesi
- Satır sonunun iki karaktere genişletilmesi
- Satır sonunun silinip yerine sayaç konması

Veri dosyası bu nedenle ikili modda açılır

Kodlama seçimi alanın boyutunu belirler

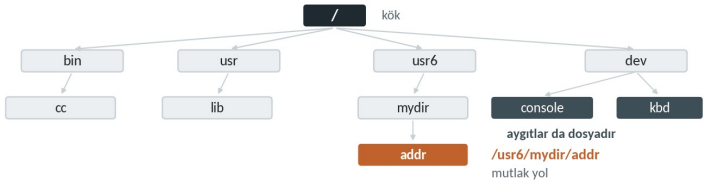
- ASCII tek bayt, UTF-8 değişken, UTF-32 sabit dört bayt

Dosya yapısı kuran programcı otomatik yardımı kapatmak zorundadır

Dosya yapısı kuran bir programcı, dosyalarında beklemediği karakterlerle karşılaşabilmektedir. Bazı sistemler dosyanın sonuna bir denetim karakteri eklemekte, bazıları satır sonunu iki karakterlik bir çifte genişletmekte, bazıları ise satır sonu karakterlerini silip yerlerine karakter sayacı koymaktadır. Bu değişiklikler metin dosyası varsayımından kaynaklanmakta ve kayıt sınırlarını bozmaktadır. Bu nedenle veri dosyaları ikili kipte açılmaktadır. Kodlama seçimi de alan boyutunu doğrudan belirlemektedir: ASCII her karakter için tek bayt, UTF-8 değişken sayıda bayt, UTF-32 sabit dört bayt kullanmaktadır.

UNIX

Dizin yapısı ve dosya soyutlaması



Göreli yol geçerli dizine göredir. Nokta geçerli dizini, iki nokta üst dizini gösterir.

Aygıtlar da dizinde görünür: /dev/kbd, /dev/console

Unix'te dosya, nasıl saklandığına bakılmaksızın bayt dizisidir

Mantıksal dosya tek bir tam sayıyla temsil edilir

Unix dosya sistemi, kökü eğik çizgiyle gösterilen ağaç yapılı bir dizin düzenidir. Dizinler hem sıradan dosyaları hem başka dizinleri içermektedir. Her dosya kökten başlayan mutlak yol adıyla tek biçimde tanımlanabilmektedir. Eğik çizgiyle başlamayan yol adları ise geçerli dizine göre yorumlanmaktadır. Nokta geçerli dizini, iki nokta üst dizini göstermektedir. Aygıtlar da dosya olarak ele alındığından dizinlerde aygıt girdileri de bulunmaktadır.

UNIX

Yönlendirme, borular ve komutlar

- Üç standart dosya: girdi, çıktı ve hata
  - Klavye, konsol ve hata çıktısı
- Yönlendirme, çalışma anında bu dosyaları değiştirir
  - < dosya girdiyi, > dosya çıktıyı yönlendirir
- Pipe, bir programın çıktısını diğerinin girdisi yapar
  - list | sort: ara dosya kullanılmaz
- Dosya komutları: cat, tail, cp, mv, rm, chmod, ls, mkdir, rmdir
- Tanımlar başlık dosyalarındadır: stdio.h, fstream.h, fcntl.h

Programların çoğu girdisini klavyeden almakta, çıktısını konsola yazmaktadır. Hata iletileri için ayrı bir dosya kullanılmaktadır. Yönlendirme, bu standart dosyaların çalışma anında başka dosyalarla değiştirilmesine olanak tanımaktadır. Küçüktür işareti girdiyi, büyüktür işareti çıktıyı yönlendirmektedir. Pipe ise bir programın çıktısını ara dosya kullanmadan başka bir programın girdisi yapmaktadır. Listeleme çıktısının sıralama programına verilmesi bunun tipik örneğidir. Bu bölümde adı geçen değerler ve bayraklar, sistemin başlık dosyalarında tanımlanmaktadır.

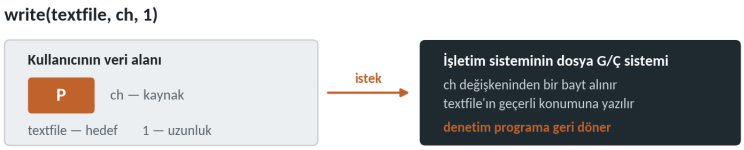
2

Bir baytın yolculuğu

Program ile disk yüzeyi arasındaki katmanlar

YOLCULUK · 1

Yolculuğun başlangıcı



Program yalnızca isteği bildirmektedir. İsteğin yerine getirilmesi işletim sisteminin sorumluluğundadır.

İstek üç bileşenden oluşur: kaynak, hedef, uzunluk

Çağrı işletim sistemine devredilir

Denetim programa hemen geri döner

Bir baytın yolculuğu, tek bir yazma çağrısıyla başlamaktadır. Örnekte bir karakter değişkeninde tutulan P harfinin, adı bir değişkende saklanan bir metin dosyasının sonuna eklenmesi istenmektedir. Çağrının üç bileşeni bulunmaktadır: verinin alınacağı kaynak, verinin yazılacağı dosya ve aktarılacak bayt sayısı. Programın görevi bu noktada bitmektedir. Çağrı işletim sistemine iletilmekte, isteğin yerine getirilmesiyle ilgili bütün kararlar işletim sistemine bırakılmakta ve denetim programa geri dönmektedir.



Yazma isteği, her biri bir öncekinden daha fiziksel olan bir dizi katmandan geçmektedir. Üstteki katmanlar dosyayı adıyla ve mantıksal yapısıyla ele almakta, alttaki katmanlar ise sürücü, silindir, iz ve sektör gibi fiziksel birimlerle çalışmaktadır. İşletim sisteminin dosyalarla ilgilenen alt kümesine dosya yöneticisi adı verilmektedir. Dosya yöneticisi önce dosyanın açık olup olmadığını, hangi erişim izinlerinin bulunduğunu ve türünün ne olduğunu denetlemekte, ardından dosya tahsis tablosundan hedef sektörün fiziksel yerini belirlemektedir.

YOLCULUK · 3

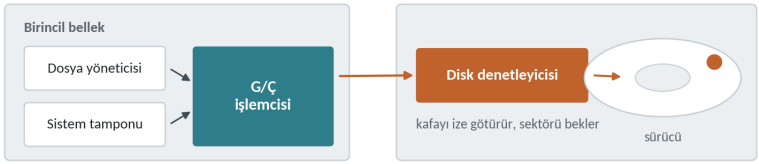
Sistem giriş çıkış tamponu

- Hedef sektör bellekte değilse önce tampona okunur
- Bayt ancak bundan sonra tampon içine yerleştirilir
- Tampon, bellekteki düzeni diskteki düzenle eşler
- Bayt çoğunlukla hemen gönderilmez
  - Aynı sektöre gidecek baytlar biriktirilir
- Dosya kapatıldığında tamponlar boşaltılmak zorundadır
  - Boşaltılmazsa yazılan veri kaybolur

Baytın yazılacağı sektörün belleğe alınmış olması gerekmektedir. Sektör bellekte değilse dosya yöneticisi boş bir sistem tamponu bulmakta ve sektörü diskten bu tampona okumaktadır. Bayt ancak bundan sonra tampon içindeki doğru konuma yerleştirilmektedir. Sistem tamponu, bellekteki düzenin diskteki düzenle aynı olmasını sağlamaktadır. Tamponun içeriği çoğunlukla hemen gönderilmemekte, aynı sektöre gidecek başka baytların birikmesi beklenmektedir. Dosya kapatıldığında ise bekleyen bütün tamponların diske boşaltılması zorunludur. Aksi durumda veri kaybı ortaya çıkmaktadır.

YOLCULUK · 4

Baytın bellekten ayrılışı



Bellek içi veri yolu geniş ve hızlıdır; diske giden yol dar ve yavaştır. Aradaki farkı yönetmek için ayrı bir işlemci kullanılmaktadır.

Dar ve yavaş yol, veriyi parçalara böler

Özelleşmiş iş ayrı bir işlemciye bırakılır

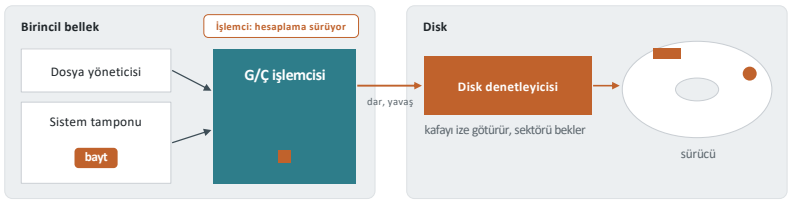
G/Ç işlemcisi bağımsız çalışır, örtüşme sağlar

17

Baytın bellekten ayrılması, hız ve genişlik bakımından farklı bir veri yoluna geçmesi anlamına gelmektedir. Bellek içindeki veri yolu geniş ve hızlıyken diske giden yol dar ve yavaştır. Bu farklılık nedeniyle veri, dışarıdaki aygıtın kaldırabileceği büyüklükte parçalara bölünerek gönderilmektedir. Bu özelleşmiş işi genel amaçlı işlemciye yaptırmak verimsiz olduğundan, işlem ayrı bir birime, giriş çıkış işlemcisine bırakılmaktadır. Giriş çıkış işlemcisi yönergelerini işletim sisteminden almakta, sonrasında bağımsız çalışarak hesaplama ile aktarımın örtüşmesini sağlamaktadır.

HAREKETLI ŞEKIL

Baytın bellekten ayrılışı



Başlangıç · bayt sistem tamponunda, aktarım bekleniyor
1 · işletim sistemi aktarımı G/Ç işlemcisine devreder
2 · dar yolda veri parçalara bölünür; işlemci kendi işine döner
3 · denetleyici kafayı hedef ize götürür, sektör gelince yazar

Bayt önce G/Ç işlemcisine, sonra parça parça denetleyiciye geçer

İşlemci bu sırada hesaplamaya devam eder

18

Bu hareketli şekil bir önceki slaytta anlatılan aktarımı adım adım göstermektedir. Başlangıçta bayt sistem tamponunda beklemektedir. Birinci tıklamada işletim sistemi aktarımı giriş çıkış işlemcisine devretmekte ve bayt bu işlemciye geçmektedir. İkinci tıklamada diske giden yolun dar ve yavaş olması nedeniyle veri parçalara bölünerek denetleyiciye gönderilmektedir. Bu sırada genel amaçlı işlemci aktarımı beklemeden kendi hesaplamasına dönmekte, böylece hesaplama ile aktarım örtüşmektedir. Üçüncü tıklamada disk denetleyicisi okuma yazma kafasını hedef ize götürmekte ve sektör kafanın altına geldiğinde veri yüzeye yazılmaktadır.



Diskin çalışmasını yöneten birime disk denetleyicisi adı verilmektedir. Giriş çıkış işlemcisi, sürücünün yazmaya hazır olup olmadığını denetleyiciye sormaktadır. Yoğun bir sistemde sürücünün hazır olmaması olasıdır. Bu durumda bayt sırası gelene kadar tamponda beklemektedir. Sıra geldiğinde denetleyici okuma yazma kafasını hedef ize taşımakta ve istenen sektörün kafanın altına gelmesini beklemektedir. Bu aşama, yolculuğun mekanik hareket içeren ilk aşamasıdır ve o ana kadar harcanan sürenin tamamını önemsiz kılmaktadır.

YOLCULUK · 6

Yolculuğun sonu

Plaka dönerken sekiz bit yüzeye tek tek bırakılır

Karakter, saatte 50 ile 100 mil hızla dönen yüzeyde kalır

Tek bir bayt için katedilen yol budur

Her katman bir soyutlama sunar

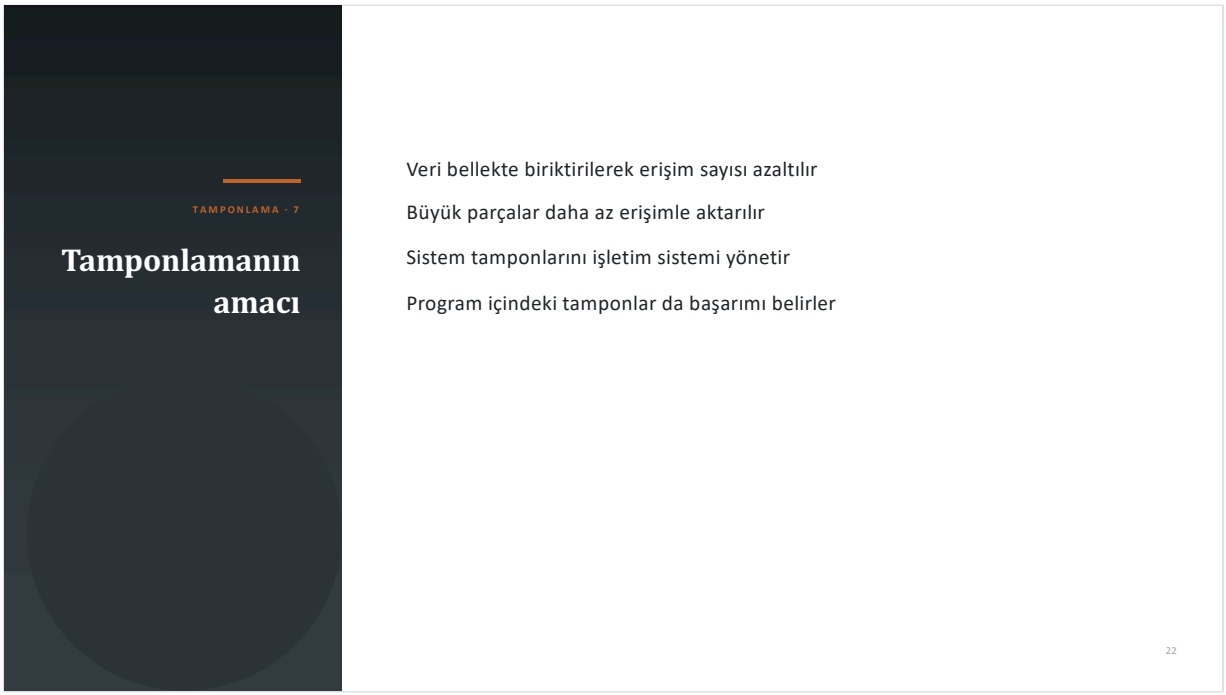
- Soyutlama programcıyı donanımdan bağımsızlaştırır
- Aynı soyutlama maliyeti de gizler

Yolculuğun son aşamasında plaka kafanın altında dönerken baytın sekiz biti yüzeye tek tek bırakılmaktadır. Karakter bu noktada, hızlı dönen yüzeyde yerini almaktadır. Tek bir baytın bu kadar uzun bir yol izlemesi, dosya tasarımında neden erişim sayısının azaltılmaya çalışıldığını açıklamaktadır. Yolculuğun her katmanı bir soyutlama sunmakta, bu soyutlamalar programcının donanım ayrıntılarından bağımsız çalışmasını sağlamakta, ancak maliyeti de gizlemektedir.

3

Tamponlama

Diskin yavaşlığının bellekle dengelenmesi



Tamponlama, program veri alanı ile ikincil bellek arasında akan verinin bellekte biriktirilmesidir. Amaç, ikincil belleğe yapılan erişim sayısını azaltmaktır. Büyük parçalarla çalışıldığında aynı miktarda veri daha az erişimle aktarılmaktadır. Program içinde kullanılan tamponların da başarım üzerinde belirleyiciliği bulunmaktadır.



Tek bir sistem tamponunun kullanıldığı bir düzende aynı anda hem girdi hem çıktı yapılması olanaksızdır. Program bir karakter okuduğunda tampon, o karakteri içeren sektörle dolmaktadır. Bir karakter yazmak istediğinde ise aynı tampon çıktı sektörüyle üzerine yazılmaktadır. Sonraki girdi isteğinde tamponun içeriğinin diske yazılması ve okunan sektörün yeniden getirilmesi gerekmektedir. Bu nedenle giriş çıkış sistemleri en az iki tampon kullanmaktadır: biri girdi, diğeri çıktı içindir.

TAMPONLAMA · 9

Giriş çıkış bağımlı çalışma

Tek yönlü aktarımda bile tek tampon yavaşlatır

Sektör okuma, bellek içi taşımaya göre çok yavaştır

İşlemci zamanının çoğunu beklemekle geçirir

- Bu duruma G/Ç bağımlı çalışma denir

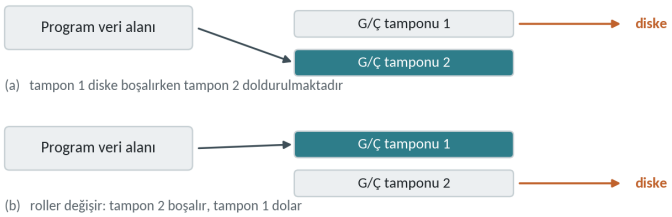
Çözüm: tampon sayısını artırmak

- Biri işlenirken diğeri doldurulur

Tek yönlü aktarımda bile tek tamponun başarımı düşürdüğü görülmektedir. Bir sektörün diskten okunması, verinin bellek içinde taşınmasına göre çok yavaştır. Dolayısıyla çok sayıda sektör okuyan bir program, her okuma isteğinden sonra tamponun dolmasını beklemek zorunda kalmaktadır. İşlemcinin zamanının büyük bölümünü beklemekle geçirdiği bu duruma giriş çıkış bağımlı çalışma adı verilmektedir. Çözüm, tampon sayısını artırmak ve işlemci bir tamponu işlerken giriş çıkış sisteminin bir sonrakini doldurmasını sağlamaktır.

TAMPONLAMA · 10

Çoklu tamponlama



(a) tampon 1 diske boşalırken tampon 2 doldurulmaktadır

(b) roller değişir: tampon 2 boşalır, tampon 1 dolar

İki tamponun rolleri her aktarımdan sonra değişir

Bu biçimine çift tamponlama denir

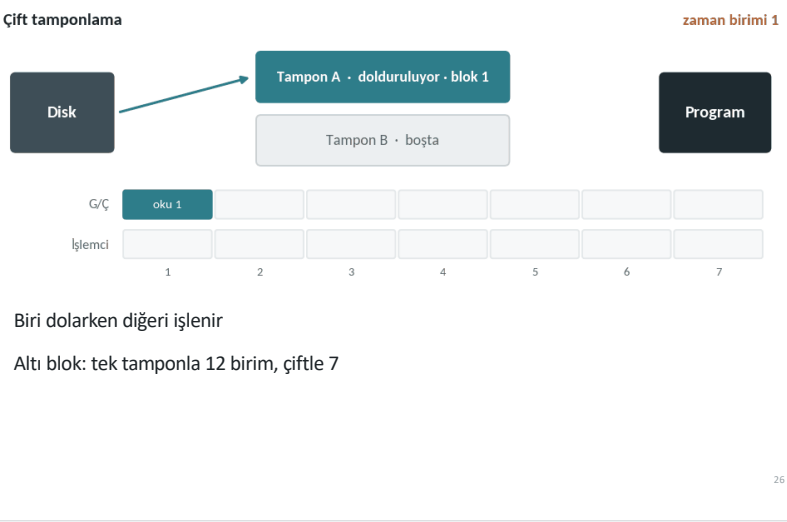
İkiden çok tamponla da uygulanabilir

25

Çoklu tamponlama, işlemcinin bir tamponu işlerken giriş çıkış sisteminin diğerini doldurması ya da boşaltması ilkesine dayanmaktadır. İki tamponun kullanıldığı ve her aktarımdan sonra rollerinin değiştirildiği biçimine çift tamponlama denilmektedir. İşlemci ile giriş çıkış birimi böylece birbirini beklemeden çalışabilmektedir. Yöntem ikiden çok tamponla da uygulanabilmektedir. Tamponların yönetimi işletim sistemi tarafından yapılmakta, sistem düzeyinde çalışmayan programcının bu ayrıntıyı ele alması gerekmemektedir. Ancak tampon sayısının ayarlanmasına çoğu sistemde izin verilmektedir.

HAREKETLI ŞEKIL

Çift tamponlama



Çift tamponlamada iki tamponun rolü her zaman biriminin sonunda değişmektedir. İlk birimde yalnızca birinci tampon doldurulmakta, işlemci henüz işleyecek veri bulamadığından beklemektedir. İkinci birimden başlayarak giriş çıkış birimi bir tamponu doldururken işlemci öbür tampondaki bloğu işlemektedir. Okuma ve işlemenin her biri bir birim sürdüğünde altı bloğun tek tamponla işlenmesi on iki birim almakta, çift tamponla ise yedi birimde tamamlanmaktadır. Kazanç, iki işin aynı anda yürütülebilmesinden gelmektedir. Okuma işlemeden belirgin biçimde yavaşsa işlemci yine beklemekte, bu durumda tampon sayısının artırılması da süreyi okuma süresinin altına indirememektedir.

TAMPONLAMA · 11

Tampon havuzu ve LRU



İstenen blok havuzdaysa diske hiç gidilmez

Yoksa kullanımda olmayan bir tampon seçilir

Seçim ölçütü en uzun süredir kullanılmamış tampondur

27

Tampon havuzu, sistem tamponlarının önceden ayrılmış bir küme içinden verilmesidir. Bir sektör ya da blok istendiğinde sistem önce havuzdaki tamponlara bakmakta, istenen blok zaten bellekteyse diske hiç gidilmemektedir. Blok bulunamazsa havuzdan kullanımda olmayan bir tampon seçilmekte ve blok bu tampona okunmaktadır. Seçim ölçütü çoğunlukla en uzun süredir kullanılmamış tampondur. Bu ölçüt, yakın geçmişte kullanılan bir bloğun yakın gelecekte yeniden istenmesinin daha olası olduğu varsayımına dayanmaktadır.

ÖRNEK

LRU ile tampon havuzu

İstek	Tampon 1	Tampon 2	Tampon 3	Sonuç	Çıkarılan
1. blok 5	5			ıska	
2. blok 2	5	2		ıska	
3. blok 5	5	2		isabet	
4. blok 7	5	2	7	ıska	
5. blok 5	5	2	7	isabet	
6. blok 3	5	3	7	ıska	2
7. blok 2	5	3	2	ıska	7
8. blok 5	5	3	2	isabet	

Sekiz istekte 3 isabet, 5 disk erişimi

Çıkarılan: en uzun süredir kullanılmayan blok

28

Örnekte üç tampondan oluşan bir havuza sırayla sekiz blok isteği gelmektedir. İlk iki istek boş tamponlara yerleşmekte, üçüncü istekte beşinci blok zaten havuzda bulunduğundan diske gidilmemektedir. Altıncı istekte havuz doludur ve üçüncü bloğa yer açmak gerekmektedir. Beşinci blok bir önceki istekte, yedinci blok dördüncü istekte kullanılmışken ikinci blok en son ikinci istekte kullanılmıştır. Bu nedenle çıkarılan blok ikinci bloktur. Yedinci istekte aynı ölçütle yedinci blok çıkarılmaktadır. Sekiz isteğin üç tanesi havuzdan karşılanmakta, beş tanesi disk erişimi gerektirmektedir.

HAREKETLI ŞEKIL

LRU ile tampon havuzu

İstek	Tampon 1	Tampon 2	Tampon 3	Sonuç	Çıkarılan
1. blok 5	5			ıska	
2. blok 2	5	2		ıska	
3. blok 5	5	2		isabet	
4. blok 7	5	2	7	ıska	
5. blok 5	5	2	7	isabet	
6. blok 3	5	3	7	ıska	2
7. blok 2	5	3	2	ıska	7
8. blok 5	5	3	2	isabet	

Sekiz istekte 3 isabet, 5 disk erişimi

Çıkarılan: en uzun süredir kullanılmayan blok

29

Bu hareketli şekilde tablo her tıklamada bir satır ilerlemektedir; her istek geldiğinde havuzun durumu, sonucun isabet mi ıska mı olduğu ve gerekiyorsa çıkarılan blok birlikte görülmektedir. Örnekte üç tampondan oluşan bir havuza sırayla sekiz blok isteği gelmektedir. İlk iki istek boş tamponlara yerleşmekte, üçüncü istekte beşinci blok zaten havuzda bulunduğundan diske gidilmemektedir. Altıncı istekte havuz doludur ve üçüncü bloğa yer açmak gerekmektedir. Beşinci blok bir önceki istekte, yedinci blok dördüncü istekte kullanılmışken ikinci blok en son ikinci istekte kullanılmıştır. Bu nedenle çıkarılan blok ikinci bloktur. Yedinci istekte aynı ölçütle yedinci blok çıkarılmaktadır. Sekiz isteğin üç tanesi havuzdan karşılanmakta, beş tanesi disk erişimi gerektirmektedir.

4

Unix'te giriş çıkış

File descriptor'dan aygıt sürücüsüne

UNIX · 12

En üst katman: prosesler

Prosesler dosyalara mantıksal görünüm yükler

Shell komutları: cat, tail

Dosyalar üzerinde çalışan kullanıcı programları

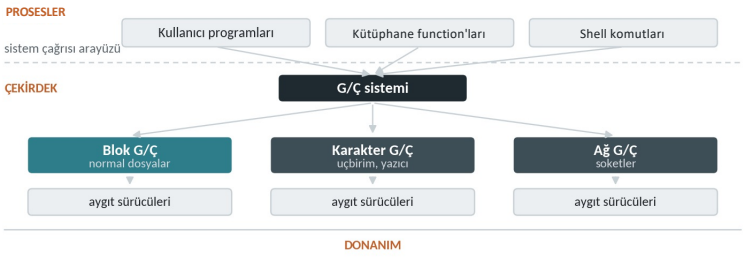
Kütüphane fonksiyonları: scanf, fread

- Dizge ve sayı okumayı sistem çağrısına çevirir

En üstteki katman veriyi mantıksal terimlerle ele almaktadır. Bu katman, dosyalara belirli mantıksal görünümler yükleyen proseslerden oluşmaktadır. Prosesler bir sorunu çözmek üzere çalışmaktadır. Dosyadaki sözcükleri saymak ya da bir adres aramak gibi işler yapmaktadır. Bu katmanda üç tür bileşen bulunmaktadır: cat ve tail gibi shell komutları, dosyalar üzerinde çalışan kullanıcı programları ve programlardan çağrılan kütüphane fonksiyonları. Kütüphane fonksiyonları, dizge ve sayı okuma gibi işleri sistem çağrılarına dönüştürmektedir.

UNIX · 13

Çekirdek

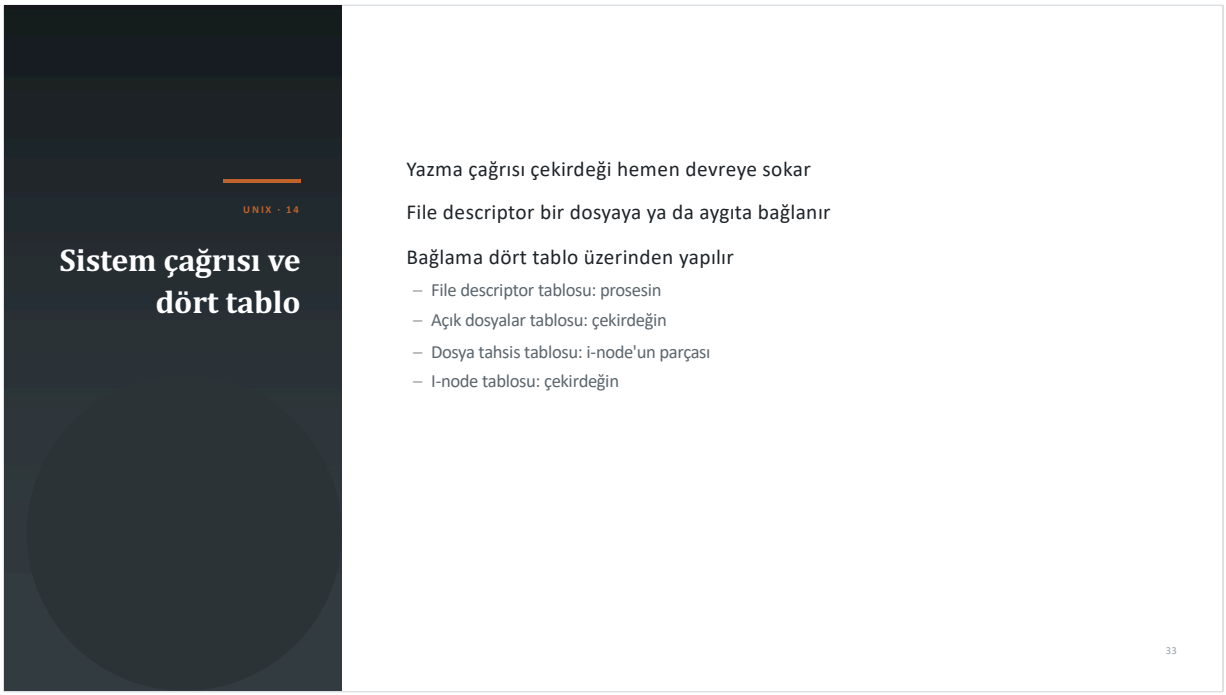


Çekirdek bütün G/Ç'yi bayt dizisi olarak görür

Mantıksal yapıya ilişkin varsayımlar burada biter

32

Bu katmanın altında, geri kalan bütün katmanları içeren Unix çekirdeği bulunmaktadır. Çekirdek bütün giriş çıkışı bir bayt dizisi üzerinde çalışıyormuş gibi görmektedir. Denetim çekirdeğe geçtiği anda dosyanın mantıksal yapısına ilişkin bütün varsayımlar ortadan kalkmaktadır. En üst katmanın altındaki bütün işlemlerin uygulamanın mantıksal görünümünden bağımsız kılınması, Unix tasarımını ayırt eden bir karardır. Bu karar, dosya yapısıyla ilgili kararların tümüyle tasarımcıya bırakılmasını sağlamaktadır. Tek koşul dosyanın bir bayt dizisinden kurulmuş olmasıdır.



Bir yazma isteği sistem çağrısı arayüzü üzerinden çekirdeğe iletilmektedir. Çekirdeğin giriş çıkış sistemi, programdaki file descriptor'ı dosya sistemindeki bir dosyaya ya da aygıta bağlamaktadır. Bu bağlama dört tablo üzerinden yapılmaktadır: file descriptor tablosu, açık dosyalar tablosu, bir i-node'un parçası olan dosya tahsis tablosu ve i-node tablosu. Tablolar çekirdeğin giriş çıkış sistemi tarafından yönetilmekle birlikte farklı bileşenlere aittir: file descriptor tablosu prosesin, açık dosyalar ve i-node tabloları çekirdeğin, i-node'un kendisi ise dosya sisteminindir.

UNIX · 15

File descriptor ve açık dosyalar tablosu



— 0 — standart girdi, 1 — standart çıktı, 2 — standart hata
— Açık dosyalar tablosu kipi, konumu ve kullanan proses sayısını tutar
— i-node dosya var olduğu sürece diskte durur, açılınca belleğe alınır

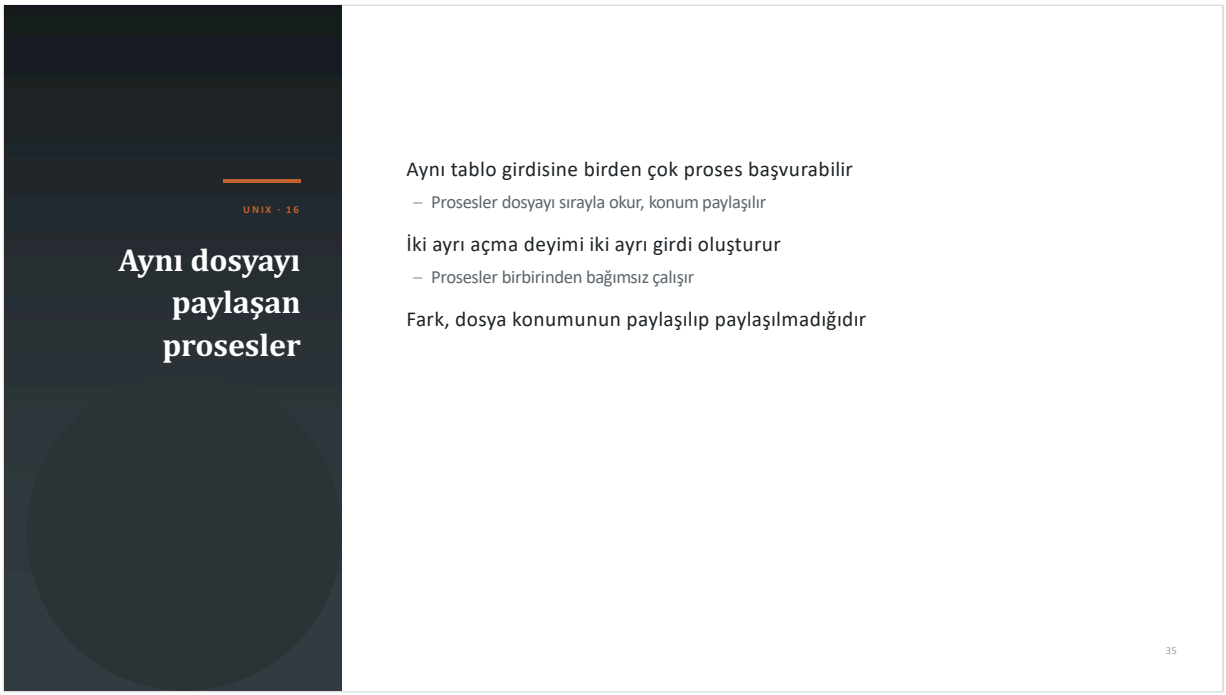
Her prosesin kendi file descriptor tablosu vardır

Açık dosyalar tablosu kipi, konumu ve proses sayısını tutar

Girdi, dosya türüne uygun function'ların adreslerini içerir

34

File descriptor tablosu, prosesin kullandığı file descriptor'ları açık dosyalar tablosundaki girdilerle eşleyen basit bir tablodur. Her prosesin kendi file descriptor tablosu bulunmakta ve bu tablo standart girdi, standart çıktı ve standart hata dosyalarını da içermektedir. Açık dosyalar tablosunda ise açılmış her dosya için bir girdi yer almaktadır. Bu girdilerde dosyanın hangi kiple açıldığı, kaç proses tarafından kullanıldığı ve bir sonraki erişimin dosya içindeki konumu tutulmaktadır. Girdi ayrıca dosya türüne uygun işlem function'larının adreslerini de içermektedir.



Aynı açık dosyalar tablosu girdisine birden çok prosesin başvurması olanaklıdır. Bu durumda prosesler dosyayı sırayla okumakta, her biri bir öncekinin bıraktığı yerden devam etmektedir. Buna karşılık aynı dosya iki ayrı açma deyimiyle açılırsa tabloya iki ayrı girdi eklenmekte ve prosesler dosya üzerinde birbirinden bağımsız çalışmaktadır. İki davranış arasındaki fark, dosya konumunun paylaşılıp paylaşılmadığıdır. Bir proses yazarken başka bir prosesin aynı dosyadan bağımsız olarak okuması, sonucun yorumlanmasını güçleştirmektedir.

UNIX · 17

Açık dosyalar tablosu geçicidir

Tablo, dosyayla ne yapılabileceğini bildirir

Dosyanın diskteki yeri bu tabloda yoktur

Boyut ve sahiplik bilgisi de bulunmaz

Bu bilgiler i-node'da tutulur

- Yaygın adıyla i-node

Açık dosyalar tablosundaki bilgi geçicidir. Bu tablo, açılmış bir dosyayla hangi işlemlerin yapılabileceğini ve bunların nasıl yapılacağını çekirdeğe bildirmektedir. Ancak çekirdeğin dosya hakkında daha fazlasını bilmesi gerekmektedir: dosyanın diskte nerede saklandığı, ne kadar büyük olduğu ve kime ait olduğu bu tabloda yer almamaktadır. Bu bilgiler index node adı verilen, yaygın kullanımıyla i-node olarak anılan yapıda tutulmaktadır.

UNIX · 18

I-node

Tablo girdisi yalnızca dosya açıkken vardır

I-node dosya var olduğu sürece vardır

Dosyayla birlikte diskte saklanır

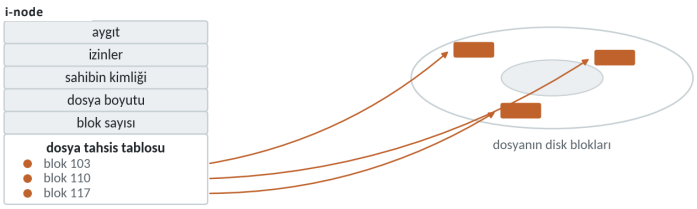
- Fiziksel olarak bitişik olması gerekmez

Dosya açılınca bir kopyası i-node tablosuna alınır

I-node, açık dosyalar tablosundaki girdiden daha kalıcı bir yapıdır. Tablo girdisi yalnızca dosya açıkken var olmakta, i-node ise dosya var olduğu sürece varlığını sürdürmektedir. Bu nedenle i-node, fiziksel olarak bitişik olmasa da dosyanın kendisiyle birlikte diskte saklanmaktadır. Dosya açıldığında i-node'un bir kopyası belleğe alınmakta ve hızlı erişim için i-node tablosuna eklenmektedir. Böylece dosya üzerindeki her işlemde diske yeniden gidilmesi gerekmemektedir.

UNIX · 19

Blok indeksi ve aygıt sürücüsü



i-node dosyanın kendisiyle birlikte diskte saklanır; en önemli bileşeni dosyayı oluşturan disk bloklarının indeksidir.

Blok listesi, dosya tahsis tablosunun karşılığıdır

Büyük dosyalar için ağaç benzeri yapı kullanılır

Veri ve aygıt türüne uygun program: aygıt sürücüsü

38

I-node'un bu ders açısından en önemli bileşeni, dosyayı oluşturan disk bloklarının listesidir. Bu liste, daha önce ele alınan dosya tahsis tablosunun Unix'teki karşılığıdır. Basit bir dizi olmak zorunda değildir. Büyük ve küçük dosyaların birlikte desteklenmesi için çoğunlukla ağaç benzeri bir yapı kullanılmaktadır. Çekirdeğin giriş çıkış sistemi i-node bilgisini elde ettiğinde, verinin türüne, yapılacak işleme ve aygıtın türüne uygun bir giriş çıkış programını çağırmaktadır. Unix'te bu programa aygıt sürücüsü adı verilmektedir.

UNIX · 20

Dosya adının bağlanması

Her başvuru bir dizinden başlar

Dizin, ad ve i-node göstericisi tutan küçük bir dosyadır

Dizinden i-node'a giden gösterici sabit bağlantıdır

Dosya açılırken i-node bu bağlantıyla belleğe alınır

Bir dosyaya yapılan her başvuru bir dizinden başlamaktadır, çünkü dosya adları dizinlerde tutulmaktadır. Dizin, aslında her dosya için bir ad ile o dosyanın diskteki i-node'una bir gösterici içeren küçük bir dosyadır. Dizinden i-node'a giden bu gösterici sabit bağlantı olarak adlandırılmaktadır ve dosya hakkındaki bütün bilgilere doğrudan erişim sağlamaktadır. Dosya açıldığında i-node'un belleğe alınması ve açık dosyalar tablosundaki girdinin kurulması bu bağlantı üzerinden yapılmaktadır.

UNIX · 21

Sabit bağlantı

Birden çok ad aynı i-node'a işaret edebilir

I-node, kaç bağlantısı olduğunu tutar

Bir ad silinince dosya silinmez

- Yalnızca bağlantı sayacı bir azalır

Dosya, son bağlantı kalkınca ortadan kalkar

Birden çok dosya adının aynı i-node'a başvurması olanaklıdır. Böylece bir dosya birden çok adla anılabilmektedir. I-node içinde kendisine kaç sabit bağlantı bulunduğunu belirten bir alan yer almaktadır. Bu düzenin sonucu önemlidir: bir dosya adı silindiğinde, aynı dosyaya başka adlar da bağlıysa dosyanın kendisi silinmemekte, yalnızca i-node'un bağlantı sayacı bir azaltılmaktadır. Dosya ancak son bağlantı da kaldırıldığında ortadan kalkmaktadır.

UNIX · 22

Sembolik bağlantı



Sembolik bağlantı i-node'a değil yol adına işaret eder

Dizini ya da başka dosya sistemindeki dosyayı gösterebilir

Hedef silinince bağlantı kırılır

41

İkinci bağlantı türü sembolik bağlantı, diğer adıyla yumuşak bağlantıdır. Sembolik bağlantı bir i-node'a değil, bir dosya adına, yani bir yol adına işaret etmektedir. Gerçek bir dosyaya doğrudan bağlanmadığından bir dizini ya da başka bir dosya sistemindeki bir dosyayı da gösterebilmektedir. Buna karşılık hedefin silinmesi durumunda bağlantı kırılmaktadır. Sembolik bağlantılar bütün Unix sistemlerinde bulunmamaktadır.

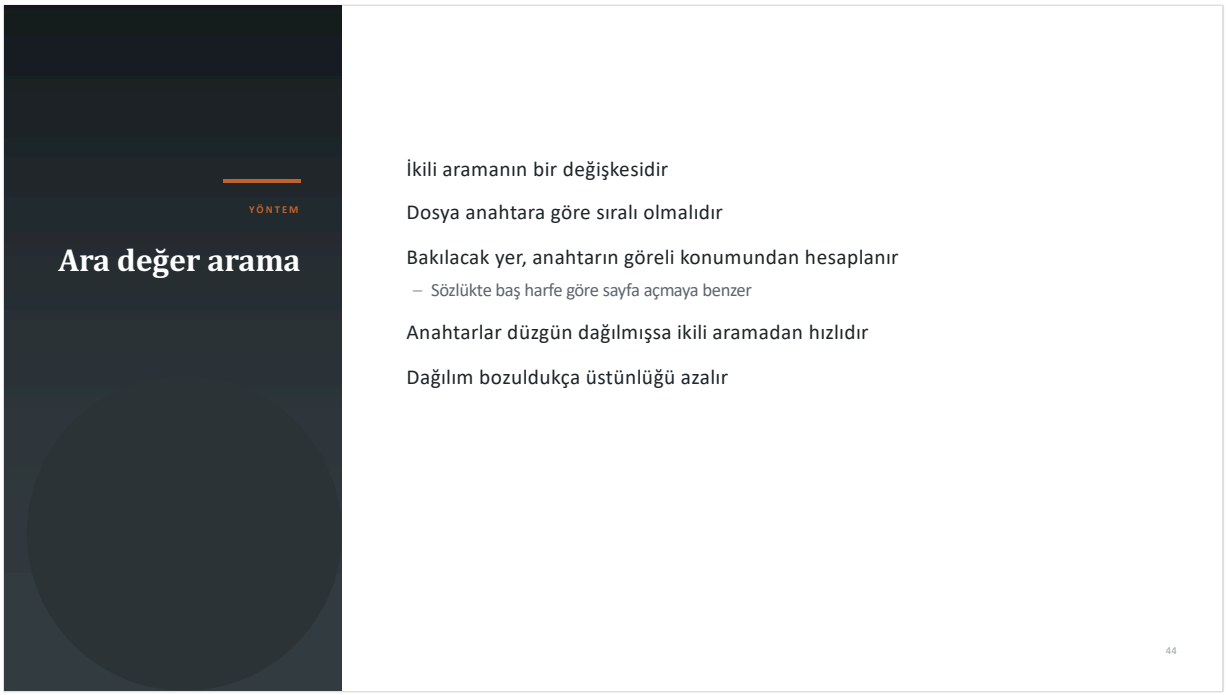
5

Algoritmalar

Ara değer arama ve kendini düzenleyen sıralı arama



Algoritma hattında ikinci haftada iki yöntem incelenmiştir. Basit sıralı arama, kayıtları baştan sona tarayarak aranan anahtarla karşılaştırmakta ve ortalama olarak kayıt sayısının yarısı kadar adım gerektirmektedir. İkili arama ise dosyanın anahtara göre sıralı olmasını koşul koşmakta ve en kötü durumda kayıt sayısının ikili logaritması kadar karşılaştırmayla sonuç vermektedir. Bu haftanın iki yöntemi de aynı çerçevede yer almakta, biri anahtarın değerinden, diğeri erişim sıklığından yararlanmaktadır.



Ara değer arama, ikili aramanın bir değişkesidir ve dosyanın anahtar değerine göre sıralı olmasını gerektirmektedir. İkili arama arama alanının ortasına bakarken, ara değer arama aranan anahtarın alt ve üst sınır arasındaki göreli konumunu hesaplayarak bakılacak yeri belirlemektedir. Yaklaşımın dayandığı sezgi, bir sözlükte aranan sözcüğün baş harfine göre sayfa açılmasıdır. Anahtarlar aralık boyunca düzgün dağılmışsa yöntem ikili aramadan daha az adım üretmekte, dağılım bozuldukça üstünlüğü azalmaktadır.

ÖRNEK

Ara değer arama · adım adım

Aranan anahtar 6 · dokuz elemanlı sıralı dizi, başlangıçta alt = 0, üst = 8

0	1	2	3	4	5	6	7	8
1	3	4	5	**6**	8	9	11	12

orta: 3 → 4

Adım	alt	üst	Hesap	orta	dizi[orta]	Karar
1	0	8	0 + (6 − 1) / (12 − 1) × (8 − 0) = 3,6 → 3	3	5	5 < 6 → alt = 4
2	4	8	4 + (6 − 6) / (12 − 6) × (8 − 4) = 4	4	6	6 = 6 → bulundu

orta = alt + ((aranan − dizi[alt]) / (dizi[üst] − dizi[alt])) × (üst − alt)

İlk hesap dizinin ortasını değil, anahtarın beklenen yerini verir

Tam sayı bölmesi aşağı yuvarlar, bu nedenle ikinci adım gerekir

Hocanın örneğinde yuvarlama yakına yapıldığı için tek adım görünür

45

Örnekte dokuz elemanlı sıralı bir dizide altı anahtarı aranmaktadır. Alt sınır sıfırıncı, üst sınır sekizinci konumdur. Formül, aranan anahtarın alt sınırdaki değere olan uzaklığını aralığın toplam genişliğine oranlamakta ve bu oranı konum aralığıyla çarpmaktadır. Elde edilen değer tam sayıya indirildiğinde üçüncü konum bulunmakta, buradaki değer arananan küçük olduğundan alt sınır dördüncü konuma çekilmektedir. İkinci adımda hesap doğrudan dördüncü konumu vermekte ve arama sonlanmaktadır. Yuvarlamanın yakına yapıldığı gerçeklemelerde aynı arama tek adımda bitmektedir. Slaytta birinci tıklama ilk adımın satırını ve yoklanan elemanı, ikinci tıklama elenen bölümü ve ikinci adımı göstermektedir.

ALGORITMA

Ara değer arama sözde kodu

```
alt = 0
ust = N - 1
while alt <= ust and dizi[alt] <= anahtar <= dizi[ust]:
    if dizi[ust] == dizi[alt]:            // bölme tanımsız
        return dizi[alt] == anahtar
    orta = alt + ((anahtar - dizi[alt]) * (ust - alt))
                / (dizi[ust] - dizi[alt])
    if   anahtar == dizi[orta]:  bulundu
    elif anahtar >  dizi[orta]:  alt = orta + 1
    else:                        ust = orta - 1
bulunamadı
```

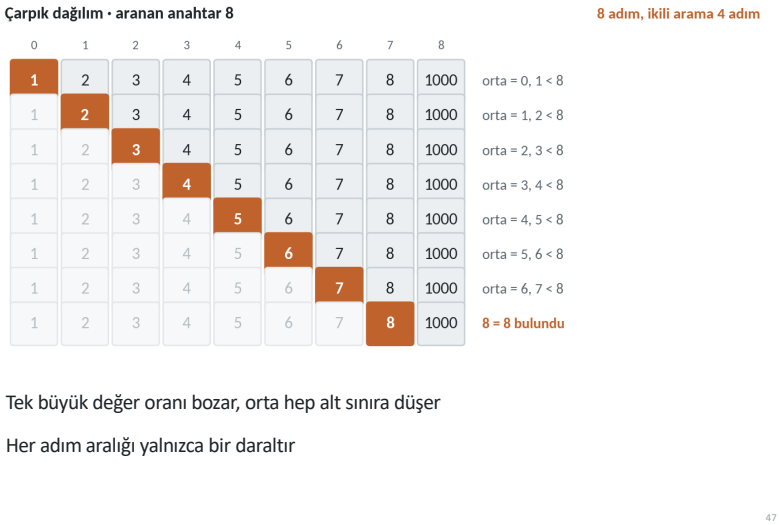
Döngü koşulu aranan anahtarı sınırlar arasında tutar

Eşit sınır değerleri ayrıca sınanmalıdır

Çarpma bölmeden önce yapılır, taşmaya dikkat edilir

SINIR

Çarpık dağılımda ara değer arama



Ara değer aramanın üstünlüğü anahtarların aralık boyunca düzgün dağılmasına bağlıdır. Dizinin son elemanı 1000 olduğunda aralığın genişliği yapay biçimde büyümekte ve 8 anahtarının göreli konumu sıfıra çok yakın hesaplanmaktadır. Formül her adımda alt sınırın kendisini vermekte, alt sınır da her adımda yalnızca bir konum ilerlemektedir. Arama bu nedenle sekiz adım sürmektedir. Aynı dizide ikili arama en çok dört adımda sonuç vermektedir. Kötü dağılımda yöntemin maliyeti kayıt sayısıyla doğru orantılı duruma gelmekte, yani sıralı aramaya yaklaşmaktadır. Uygulamada bu nedenle ara değer arama, dağılımı bilinen ve düzgün olan anahtarlarda tercih edilmektedir.



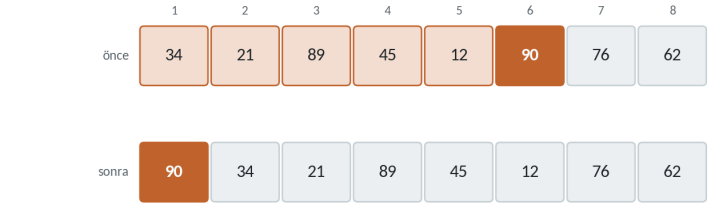
Kendini düzenleyen sıralı arama, dosyadaki bazı kayıtlara diğerlerinden daha sık erişildiği durumlarda üstünlük sağlamaktadır. Yöntemin dayandığı düşünce, sık erişilen kayıtların dosyanın başına doğru taşınmasıdır. Telefon numaralarının tutulduğu bir dosyada bazı numaraların sürekli aranması bu duruma örnektir. Yöntemin üç uygulanış biçimi bulunmaktadır: aranan kaydın başa taşınması, bir önceki kayıtla yer değiştirmesi ve erişim sayaçlarına göre sıralanması. Üçünde de sıralama ölçütü anahtarın değeri değil, erişim geçmişidir.

ADIM 1 / 6

Öne taşıma: 90 aranıyor

Öne taşıma · 1. arama: 90

6 adım · toplam 6



90, altıncı konumda: 6 adım

Başa alınır, öndeki beş kayıt kaydırılır

ADIM 2 / 6

Öne taşıma: 89 aranıyor

Öne taşıma · 2. arama: 89

4 adım · toplam 10



89, dörtüncü konumda: 4 adım

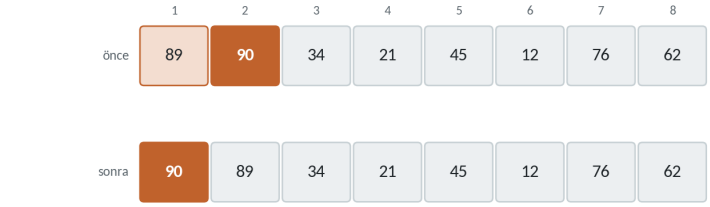
Başa alınır, öndeki üç kayıt kaydırılır

ADIM 3 / 6

Öne taşıma: 90 aranıyor

Öne taşıma · 3. arama: 90

2 adım · toplam 12



90, ikinci konumda: 2 adım

Başa alınır, öndeki bir kayıt kaydırılır

ADIM 4 / 6

Öne taşıma: 21 aranıyor

Öne taşıma · 4. arama: 21

4 adım · toplam 16



21, dörtüncü konumda: 4 adım

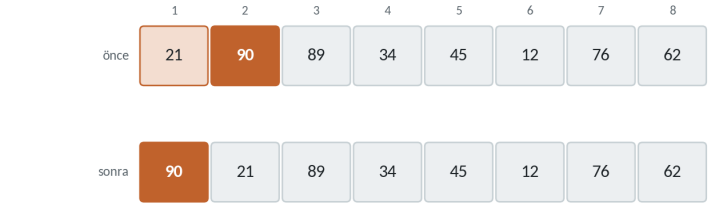
Başa alınır, öndeki üç kayıt kaydırılır

ADIM 5 / 6

Öne taşıma: 90 aranıyor

Öne taşıma · 5. arama: 90

2 adım · toplam 18



90, ikinci konumda: 2 adım

Başa alınır, öndeki bir kayıt kaydırılır

ADIM 6 / 6

Öne taşıma: 90 aranıyor

Öne taşıma · 6. arama: 90

1 adım · toplam 19



90, birinci konumda: 1 adım

Kayıt zaten başta, düzenleme yok

ÖZET

Öne taşıma

Öne taşıma (move to front)

Aranan	Aramadan önce	Adım	Aramadan sonra
90	34, 21, 89, 45, 12, 90, 76, 62	6	90, 34, 21, 89, 45, 12, 76, 62
89	90, 34, 21, 89, 45, 12, 76, 62	4	89, 90, 34, 21, 45, 12, 76, 62
90	89, 90, 34, 21, 45, 12, 76, 62	2	90, 89, 34, 21, 45, 12, 76, 62
21	90, 89, 34, 21, 45, 12, 76, 62	4	21, 90, 89, 34, 45, 12, 76, 62
90	21, 90, 89, 34, 45, 12, 76, 62	2	90, 21, 89, 34, 45, 12, 76, 62
90	90, 21, 89, 34, 45, 12, 76, 62	1	90, 21, 89, 34, 45, 12, 76, 62

Toplam adım sayısı: 19

Tek erişim kaydı doğrudan başa taşır

55

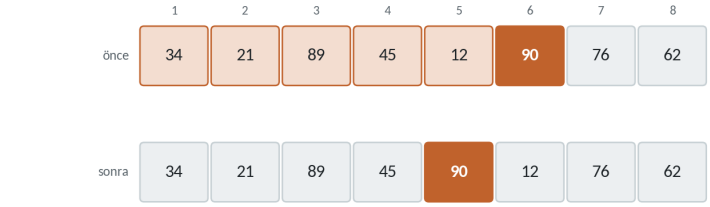
Öne taşımada aranan kayıt ilk kayıttan başlanarak sırayla aranmakta, bulunduğunda dosyanın ilk kaydı yapılmaktadır. Aranan kaydın eski konumuna kadar olan kayıtlar bir konum sonraya kaydırılmaktadır. Aranan kayıt zaten ilk kayıtsa değişiklik yapılmamaktadır. Örnekte sekiz kayıtlık bir liste üzerinde altı arama yapılmakta ve toplam adım sayısı on dokuza inmektedir. Aynı aramalar düzenleme yapılmadan yürütülseydi yirmi dokuz adım gerekecekti. Yöntemin belirgin özelliği, tek bir erişimin kaydı doğrudan başa taşıması ve bu nedenle hızlı tepki vermesidir.

ADIM 1 / 6

Konum değiştirme: 90 aranıyor

Konum değiştirme · 1. arama: 90

6 adım · toplam 6



90, altıncı konumda: 6 adım

Önündeki 12 ile yer değiştirir

ADIM 2 / 6

Konum değiştirme: 89 aranıyor

Konum değiştirme · 2. arama: 89

3 adım · toplam 9



89, üçüncü konumda: 3 adım

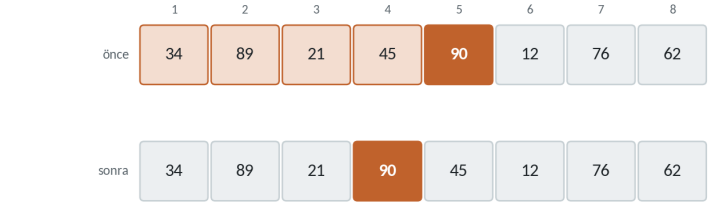
Önündeki 21 ile yer değiştirir

ADIM 3 / 6

Konum değiştirme: 90 aranıyor

Konum değiştirme · 3. arama: 90

5 adım · toplam 14



90, beşinci konumda: 5 adım

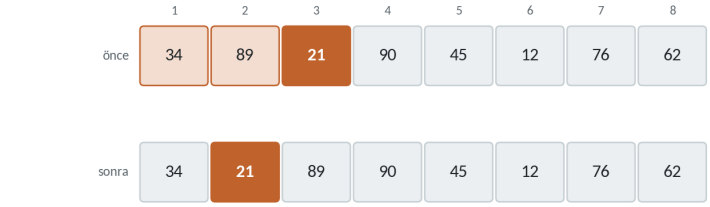
Önündeki 45 ile yer değiştirir

ADIM 4 / 6

Konum değiştirme: 21 aranıyor

Konum değiştirme · 4. arama: 21

3 adım · toplam 17



21, üçüncü konumda: 3 adım

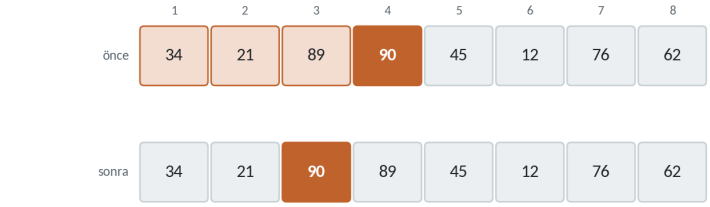
Önündeki 89 ile yer değiştirir

ADIM 5 / 6

Konum değiştirme: 90 aranıyor

Konum değiştirme · 5. arama: 90

4 adım · toplam 21



90, dörtüncü konumda: 4 adım

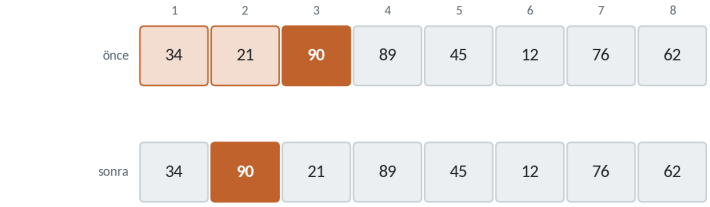
Önündeki 89 ile yer değiştirir

ADIM 6 / 6

Konum değiştirme: 90 aranıyor

Konum değiştirme · 6. arama: 90

3 adım · toplam 24



90, üçüncü konumda: 3 adım

Öndeki 21 ile yer değiştirir

ÖZET

Konum değiştirme

Konum değiştirme (transpose)

Aranan	Aramadan önce	Adım	Aramadan sonra
90	34, 21, 89, 45, 12, 90, 76, 62	6	34, 21, 89, 45, 90, 12, 76, 62
89	34, 21, 89, 45, 90, 12, 76, 62	3	34, 89, 21, 45, 90, 12, 76, 62
90	34, 89, 21, 45, 90, 12, 76, 62	5	34, 89, 21, 90, 45, 12, 76, 62
21	34, 89, 21, 90, 45, 12, 76, 62	3	34, 21, 89, 90, 45, 12, 76, 62
90	34, 21, 89, 90, 45, 12, 76, 62	4	34, 21, 90, 89, 45, 12, 76, 62
90	34, 21, 90, 89, 45, 12, 76, 62	3	34, 90, 21, 89, 45, 12, 76, 62

Toplam adım sayısı: 24

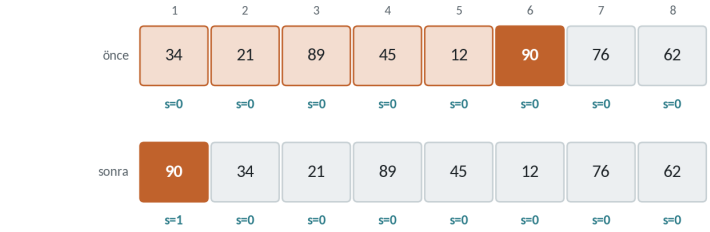
Kayıt başa doğru adım adım ilerler

ADIM 1 / 6

Sayaç: 90 aranıyor

Sayaç · 1. arama: 90

6 adım · toplam 6



90, altıncı konumda: 6 adım

Sayaç 1 oldu, birinci konuma geçer

ADIM 2 / 6

Sayaç: 89 aranıyor

Sayaç · 2. arama: 89

4 adım · toplam 10



89, dörtüncü konumda: 4 adım

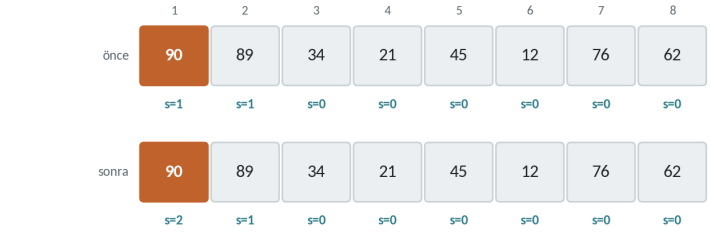
Sayaç 1 oldu, ikinci konuma geçer

ADIM 3 / 6

Sayaç: 90 aranıyor

Sayaç · 3. arama: 90

1 adım · toplam 11



90, birinci konumda: 1 adım

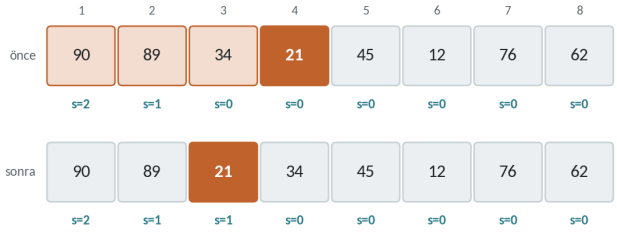
Sayaç 2 oldu, sıra değişmez

ADIM 4 / 6

Sayaç: 21 aranıyor

Sayaç · 4. arama: 21

4 adım · toplam 15



21, dörtüncü konumda: 4 adım

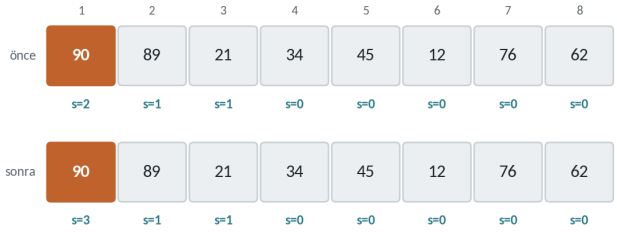
Sayaç 1 oldu, üçüncü konuma geçer

ADIM 5 / 6

Sayaç: 90 aranıyor

Sayaç · 5. arama: 90

1 adım · toplam 16



90, birinci konumda: 1 adım

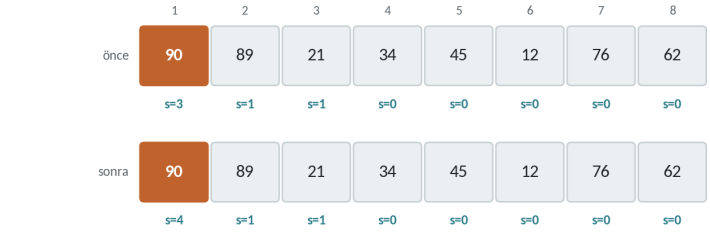
Sayaç 3 oldu, sıra değişmez

ADIM 6 / 6

Sayaç: 90 aranıyor

Sayaç · 6. arama: 90

1 adım · toplam 17



90, birinci konumda: 1 adım

Sayaç 4 oldu, sıra değişmez

ÖZET

Sayaç

Sayaç (count)

Aranan	Aramadan önce	Adım	Aramadan sonra
90	34, 21, 89, 45, 12, 90, 76, 62 sayaç: 0, 0, 0, 0, 0, 0, 0, 0	6	90, 34, 21, 89, 45, 12, 76, 62 sayaç: 1, 0, 0, 0, 0, 0, 0, 0
89	90, 34, 21, 89, 45, 12, 76, 62 sayaç: 1, 0, 0, 0, 0, 0, 0, 0	4	90, 89, 34, 21, 45, 12, 76, 62 sayaç: 1, 1, 0, 0, 0, 0, 0, 0
90	90, 89, 34, 21, 45, 12, 76, 62 sayaç: 1, 1, 0, 0, 0, 0, 0, 0	1	90, 89, 34, 21, 45, 12, 76, 62 sayaç: 2, 1, 0, 0, 0, 0, 0, 0
21	90, 89, 34, 21, 45, 12, 76, 62 sayaç: 2, 1, 0, 0, 0, 0, 0, 0	4	90, 89, 21, 34, 45, 12, 76, 62 sayaç: 2, 1, 1, 0, 0, 0, 0, 0
90	90, 89, 21, 34, 45, 12, 76, 62 sayaç: 2, 1, 1, 0, 0, 0, 0, 0	1	90, 89, 21, 34, 45, 12, 76, 62 sayaç: 3, 1, 1, 0, 0, 0, 0, 0
90	90, 89, 21, 34, 45, 12, 76, 62 sayaç: 3, 1, 1, 0, 0, 0, 0, 0	1	90, 89, 21, 34, 45, 12, 76, 62 sayaç: 4, 1, 1, 0, 0, 0, 0, 0

Toplam adım sayısı: 17

Eşit sayaçta göreli sıra korunur

69

Sayaç yönteminde her kayıt için kendisine kaç kez erişildiğini tutan bir alan bulunmaktadır. Kayıtlar sayaç değerine göre büyükten küçüğe sıralı durmaktadır. Aranan kayıt bulunduktan sonra sayacı bir artırılmakta ve gerekiyorsa kayıt yeni değerine uygun konuma taşınmaktadır. Eşit sayaç değerine sahip kayıtların göreli sırası korunmaktadır. Örnekte toplam adım sayısı on yedidir. Yöntem erişim sıklığını doğrudan ölçtüğünden en kararlı sıralamayı vermekte, buna karşılık her kayıt için ek bir alan ve her aramada bir güncelleme gerektirmektedir.

6

Uygulama · C++

Dosya işlemleri ve algoritmaların kodlanması

DOSYA · AÇMA

Açma kipleri, kapatma ve boşaltma

```
#include <fstream>
using namespace std;

struct Kayit {                  // hafta 2'de tanımlandı
    array<char, 21> ad;
    int             no;
    double          puan;
};

ofstream y("kayit.dat", ios::binary);              // yaz
ifstream o("kayit.dat", ios::binary);              // oku
fstream  d("kayit.dat", ios::in | ios::out | ios::binary);
ofstream g("gunluk.dat", ios::binary | ios::app);  // sona ekle

if (!d) { cerr << "dosya açılamadı\n"; return 1; }
d.flush();                      // tamponu diske boşaltır
d.close();                      // bağı koparır ve boşaltır
```

Üç akım sınıfı da aynı kip bayraklarını kullanır

Açmanın başarısı akımın kendisi sınanarak denetlenir

Dosya işlemlerinin C++ karşılığı akım sınıflarıdır. Bir akım nesnesi oluşturulurken dosya adı ve erişim kipi verilmektedir. Veri dosyaları her zaman ikili kipte açılmalıdır. Metin kipinde satır sonu karakterleri dönüştürüldüğünden kayıt boyutları bozulmaktadır. Ekleme kipi yazma konumunu dosyanın sonuna yerleştirmektedir. Açma işleminin başarılı olup olmadığı nesnenin kendisi sınanarak denetlenmektedir. Kapatma, mantıksal dosya ile fiziksel dosya arasındaki bağı koparmakta ve bekleyen tamponları diske boşaltmaktadır. Bu nedenle kapatılmayan bir dosyada son yazılan veriler kaybolabilmektedir.

DOSYA · KONUM

Konumlandırma ve dosya sonu

```
const auto B = sizeof(Kayit);

d.seekg(12 * B);                  // 13. kaydın başına
d.read(reinterpret_cast<char*>(&k), B);

d.seekp(12 * B);                  // yazma konumu ayrıdır
d.write(reinterpret_cast<const char*>(&k), B);

d.seekg(0, ios::end);
long boyut = d.tellg();
long n     = boyut / B;           // kayıt sayısı

d.clear(); d.seekg(0);            // bayrakları temizle
while (d.read(reinterpret_cast<char*>(&k), B))
    ++sayac;                      // okuma sonucu sınanır
```

Konum, sıra numarası ile kayıt boyutunun çarpımıdır

Dosya boyutu kayıt boyutuna bölünerek kayıt sayısı bulunur

Konumlandırma işlemleri, dosyanın herhangi bir kaydına tarama yapmadan gitmeyi sağlamaktadır. Okuma akımının konumu ile yazma akımının konumu ayrı ayrı yönetilmektedir. Konum, kaydın sıra numarasının kayıt boyutuyla çarpımından elde edilmektedir. Dosyanın sonuna gidilip geçerli konum okunduğunda dosyanın bayt cinsinden boyutu elde edilmekte, bu değer kayıt boyutuna bölünerek kayıt sayısı bulunmaktadır. Okuma döngüsü, okuma isteğinin sonucunun sınanması üzerine kurulmalıdır. Dosya sonu göstergesinin döngü koşulunda okuma öncesinde sınanması yaygın bir hata biçimidir.

ÇÖZÜMLÜ ÖRNEK

Beş kayıt: yaz, güncelle, say

```
const auto B = sizeof(Kayit);            // 40 bayt
{   ofstream y("kayit.dat", ios::binary); // v: 5 kayıt
    for (const auto& k : v)
        y.write(reinterpret_cast<const char*>(&k), B);
}                                        // kapanır, boşalır
fstream d("kayit.dat", ios::in | ios::out | ios::binary);
d.seekg(2 * B);                          // 3. kayıt
d.read(reinterpret_cast<char*>(&k), B);
cout << "3. kayit: " << k.ad.data() << ' ' << k.puan << '\n';
k.puan = 95.0;  d.seekp(2 * B);          // aynı yere
d.write(reinterpret_cast<const char*>(&k), B);
d.seekg(0, ios::end);
cout << "boyut: " << d.tellg() << " bayt, "
     << d.tellg() / B << " kayit\n";
d.clear(); d.seekg(0);
while (d.read(reinterpret_cast<char*>(&k), B))
    cout << k.no << ' ' << k.ad.data() << ' ' << k.puan << '\n';
```

ÇIKTI

```
3. kayit: Zeynep 90
boyut: 200 bayt, 5 kayit
101 Ayse 84.5
102 Mehmet 71
103 Zeynep 95
104 Ali 65.5
105 Deniz 78
```

Güncelleme yerinde, dosya 200 baytta kalır

Örnekte beş kayıtlık bir veri dosyası oluşturulmakta, üçüncü kayıt doğrudan konumlandırmayla okunup güncellenmekte ve dosya yeniden taranmaktadır. Yazma akımı bir blok içinde tanımlandığından blok sonunda kapanmakta ve tampondaki baytlar diske boşaltılmaktadır. Üçüncü kaydın konumu iki ile kayıt boyutunun çarpımıdır, çünkü sıra numaraları sıfırdan başlamaktadır. Güncellenen kayıt aynı konuma geri yazıldığından dosyanın boyutu değişmemekte, 200 baytta kalmaktadır. Dosya boyutunun kayıt boyutuna bölünmesi kayıt sayısını vermektedir. Son tarama, değişikliğin yalnızca üçüncü kayda yansıdığını göstermektedir.

ARAMA · DOSYA

Dosya üzerinde sıralı arama

```
bool sirali_ara(fstream& d, int anahtar, Kayit& k, int& adim) {
    d.clear(); d.seekg(0);            // başa dön
    adim = 0;
    while (d.read(reinterpret_cast<char*>(&k), sizeof k)) {
        ++adim;
        if (k.no == anahtar) return true;
    }
    return false;                     // dosya sonuna gelindi
}
```

Kayıtlar belleğe topluca alınmaz, dosyadan okunur

Adım sayacı okunan kayıt sayısını verir

Başarısız arama bütün dosyayı okur

Bu gerçeklemede ikinci haftada tanımlanan kayıt yapısı kullanılmaktadır. Arama, kayıtlar belleğe topluca alınmadan doğrudan dosya üzerinde yapılmaktadır. Akım önce başa konumlandırılmakta, ardından kayıtlar sırayla okunmaktadır. Döngü koşulu okuma isteğinin sonucunu sınamakta, böylece dosya sonu ayrıca denetlenmemektedir. Tutulan adım sayacı okunan kayıt sayısını vermekte ve yöntemlerin karşılaştırılmasına olanak tanımaktadır. Başarısız bir aramada dosyanın tamamının okunduğu bu kodda açıkça görülmektedir.

ARAMA · DOSYA

Dosya üzerinde ikili arama

```
bool ikili_ara(fstream& d, int anahtar, long n, Kayit& k, int& adim) {
    long alt = 0, ust = n - 1;
    adim = 0;
    while (alt <= ust) {
        long orta = alt + (ust - alt) / 2;   // taşmaya karşı
        d.seekg(orta * sizeof(Kayit));       // tarama yok
        d.read(reinterpret_cast<char*>(&k), sizeof k);
        ++adim;
        if      (k.no == anahtar) return true;
        else if (k.no <  anahtar) alt = orta + 1;
        else                      ust = orta - 1;
    }
    return false;
}
```

Her adımda tek kayıt okunur, ama her adım ayrı bir erişimdir

Aynı dosyada iki yöntemin adım sayısı karşılaştırılır

75

İkili arama, her adımda hesapladığı konuma doğrudan giderek yalnızca bir kayıt okumaktadır. Orta konum hesabında iki sınırın toplamı yerine farkı kullanılmakta, böylece büyük dosyalarda taşma önlenmektedir. Konumlandırma işlemi kayıt sıra numarasının kayıt boyutuyla çarpımı üzerinden yapılmakta ve arada kalan kayıtlar hiç okunmamaktadır. Karşılaştırma sayısı sıralı aramaya göre belirgin biçimde azalmakla birlikte her adımın ayrı bir erişim ürettiği unutulmamalıdır. İki yöntemin maliyeti aynı dosya üzerinde adım sayacıyla ölçülerek karşılaştırılmaktadır.

ÇÖZÜMLÜ ÖRNEK

İki aramanın okuma sayısı

```
// no alanı: 10, 20, ..., 150 (15 kayıt, sıralı)
for (int i = 1; i <= 15; ++i) {
    Kayit k = yap("kayit", i * 10, 0);
    y.write(reinterpret_cast<const char*>(&k), sizeof k);
}
for (int anahtar : {130, 75}) {
    bool s = sirali_ara(d, anahtar, k, adim);
    cout << anahtar << " sirali: " << (s ? "var" : "yok")
         << ", " << adim << " okuma\n";
    bool b = ikili_ara(d, anahtar, 15, k, adim);
    cout << anahtar << " ikili : " << (b ? "var" : "yok")
         << ", " << adim << " okuma\n";
}
```

ÇIKTI

```
130 sirali: var, 13 okuma
130 ikili : var, 4 okuma
75 sirali: yok, 15 okuma
75 ikili : yok, 4 okuma
```

Başarısız sıralı arama bütün dosyayı okur

İkili arama her iki durumda en çok dört okuma

76

Örnekte numara alanı onar artan on beş kayıtlık sıralı bir dosya kurulmakta ve iki anahtar her iki yöntemle aranmaktadır. Sıralı arama 130 anahtarına on üçüncü kayıtta ulaşmakta, dosyada bulunmayan 75 anahtarı için ise on beş kaydın tamamını okumaktadır. İkili arama 130 anahtarında sırasıyla sekizinci, on ikinci, on dördüncü ve on üçüncü kayıtlara bakmakta ve dört okumayla sonuca ulaşmaktadır. Başarısız aramada da okuma sayısı dörtte kalmaktadır, çünkü on beş kayıtlık bir dosyada en kötü durum dört karşılaştırmadır. Bununla birlikte ikili aramanın her okuması ayrı bir konumlandırma gerektirmekte, sıralı aramanın okumaları ise ardışık bloklardan karşılanabilmektedir.

ALGORITMA · KOD

Ara değer aramanın gerçeklenmesi

```
int ara_deger_ara(const vector<int>& a, int anahtar, int& adim) {
    int alt = 0, ust = static_cast<int>(a.size()) - 1;
    adim = 0;
    while (alt <= ust && anahtar >= a[alt] && anahtar <= a[ust]) {
        ++adim;
        if (a[ust] == a[alt])                // bölme tanımsız
            return (a[alt] == anahtar) ? alt : -1;
        long long pay = 1LL * (anahtar - a[alt]) * (ust - alt);
        int orta = alt + static_cast<int>(pay / (a[ust] - a[alt]));
        if      (a[orta] == anahtar) return orta;
        else if (a[orta] <  anahtar) alt = orta + 1;
        else                         ust = orta - 1;
    }
    return -1;
}
```

İkili aramadan yalnızca orta hesabı ve döngü koşulu farklıdır

Çarpma bölmeden önce yapılır, pay geniş tipte tutulur

77

Ara değer aramanın kodu, ikili aramadan yalnızca orta konumun hesaplanışı ve döngü koşuluyla ayrılmaktadır. Hesapta bölme yapıldığından, alt ve üst sınırdaki anahtarların eşit olması durumu ayrıca sınanmalıdır. Kesirli işlemden kaçınmak için pay ve payda tam sayı olarak düzenlenmekte, çarpma bölmeden önce yapılmaktadır. Döngü koşulunun aranan anahtarı sınır değerleri arasında tutması, anahtarın aralık dışında kalması durumunda aramanın hemen sonlanmasını sağlamaktadır. Kodun geri kalanı ikili aramayla aynı kalmaktadır.

ÇÖZÜMLÜ ÖRNEK

Düzgün ve çarpık dağılım

```
vector<int> duzgun = {1, 3, 4, 5, 6, 8, 9, 11, 12};
vector<int> carpik = {1, 2, 3, 4, 5, 6, 7, 8, 1000};
int adim;
int i = ara_deger_ara(duzgun, 6, adim);
cout << "duzgun, 6 : konum " << i << ", " << adim << " adim\n";
i = ara_deger_ara(carpik, 8, adim);
cout << "carpik, 8 : konum " << i << ", " << adim << " adim\n";
i = ara_deger_ara(carpik, 10, adim);
cout << "carpik, 10: konum " << i << ", " << adim << " adim\n";
```

ÇIKTI

```
duzgun, 6 : konum 4, 2 adim
carpik, 8 : konum 7, 8 adim
carpik, 10: konum -1, 8 adim
```

Düzgün dizide iki adım, çarpık dizide sekiz

Bulunmayan anahtar da aynı yavaşlıkta sonuçlanır

78

Örnekte aynı işlev iki farklı dizi üzerinde çalıştırılmaktadır. Düzgün dağılmış dizide 6 anahtarı iki adımda bulunmaktadır. İlk adımda tam sayı bölmesi aşağı yuvarladığından üçüncü konuma bakılmakta, ikinci adımda dördüncü konum doğrudan hesaplanmaktadır. Son elemanı 1000 olan çarpık dizide 8 anahtarı için hesaplanan konum her adımda alt sınırın kendisi olmakta, arama sekiz adım sürmektedir. Dizide bulunmayan 10 anahtarı da sekiz adım sonra, alt sınır son elemana ulaştığında döngü koşulu bozulduğu için başarısız olarak sonuçlanmaktadır. Aynı dizilerde ikili arama en çok dört adım gerektirmektedir.

ALGORITMA · KOD

Öne taşıma ve konum değiştirme

```
// öne taşıma: bulunan kayıt başa alınır, öncekiler kaydırılır
int one_tasi(vector<Kayit>& v, int anahtar) {
    for (size_t i = 0; i < v.size(); ++i) {
        if (v[i].no != anahtar) continue;
        Kayit bulunan = v[i];
        for (size_t j = i; j > 0; --j) v[j] = v[j - 1];
        v[0] = bulunan;
        return static_cast<int>(i) + 1;      // adım sayısı
    }
    return -1;
}

// konum değiştirme: yalnızca iki kayıt takas edilir
int konum_degistir(vector<Kayit>& v, int anahtar) {
    for (size_t i = 0; i < v.size(); ++i) {
        if (v[i].no != anahtar) continue;
        if (i > 0) swap(v[i - 1], v[i]);
        return static_cast<int>(i) + 1;
    }
    return -1;
}
```

Kaydırma döngüsü, dosyadaki yazma maliyetini görünür kılar

Kendini düzenleyen aramanın biçimleri aynı iskelet üzerine kurulmaktadır: kayıt sırayla aranmakta, bulunduğunda liste üzerinde bir düzenleme yapılmaktadır. Öne taşımada aranan kayıt alınmakta, önündeki kayıtlar bir konum kaydırılmakta ve kayıt başa yazılmaktadır. Konum değiştirmede yalnızca iki kayıt takas edilmektedir. İki işlev arasındaki fark kodda doğrudan görülmektedir: birinde bir kaydırma döngüsü, diğerinde tek bir takas bulunmaktadır. Dosya üzerinde çalışıldığında bu düzenlemelerin her biri ayrı bir yazma işlemine karşılık gelmekte, kaydırma döngüsünün maliyeti böylece somutlaşmaktadır.

ALGORITMA · KOD

Sayaç yönteminin gerçeklenmesi

```
struct KayitS { array<char, 21> ad; int no; double puan; int sayac; };

int sayac_ara(vector<KayitS>& v, int anahtar) {
    for (size_t i = 0; i < v.size(); ++i) {
        if (v[i].no != anahtar) continue;
        int adim = static_cast<int>(i) + 1;
        ++v[i].sayac;
        size_t j = i;                       // yeni yerine taşı
        while (j > 0 && v[j - 1].sayac < v[j].sayac) {
            swap(v[j - 1], v[j]);
            --j;                            // eşitlikte durur
        }
        return adim;
    }
    return -1;
}
```

Koşuldaki kesin küçüklük, eşit sayaçta sırayı korur

Sayaç alanı kayıt boyutunu büyütür

Sayaç yönteminde kayıt yapısına bir erişim sayacı eklenmektedir. Kayıt bulunduğunda sayacı bir artırılmakta ve yeni değerine uygun konuma taşınmaktadır. Taşıma, eklemeli sıralamanın tek adımına benzemektedir. Kayıt kendisinden önce gelen ve sayacı daha küçük olan kayıtlarla yer değiştirerek ilerlemektedir. Döngü koşulunda kesin küçüklük sınanmakta, böylece sayaçları eşit olan kayıtların göreli sırası korunmaktadır. Yöntemin bedeli, her kayıt için ayrılan ek alan ve her aramada yapılan güncellemedir.

ÇÖZÜMLÜ ÖRNEK

Üç yöntem, aynı aramalar

```
const int bas[] = {34, 21, 89, 45, 12, 90, 76, 62};
const int aramalar[] = {90, 89, 90, 21, 90, 90};
// a, b, c: bas dizisinden kurulan üç ayrı kopya
int ta = 0, tb = 0, tc = 0;
for (int x : aramalar) {
    ta += one_tasi(a, x);
    tb += konum_degistir(b, x);
    tc += sayac_ara(c, x);
}
cout << "one tasima : "; yaz(a); cout << "| " << ta << " adim\n";
cout << "konum degis: "; yaz(b); cout << "| " << tb << " adim\n";
cout << "sayac      : "; yaz(c); cout << "| " << tc << " adim\n";
```

ÇIKTI

```
one tasima : 90 21 89 34 45 12 76 62 | 19 adim
konum degis: 34 90 21 89 45 12 76 62 | 24 adim
sayac      : 90 89 21 34 45 12 76 62 | 17 adim
```

Adım sayıları elle izlenen tablolarla aynı: 19, 24, 17

Son sıralar yöntemin karakterini yansıtır

Örnekte algoritma hattında elle izlenen altı arama, üç işlevle ayrı ayrı çalıştırılmaktadır. Program çıktısındaki toplam adım sayıları, öne taşıma için 19, konum değiştirme için 24 ve sayaç yöntemi için 17 olarak adım adım izlemeyle aynı çıkmaktadır. Son sıralar da yöntemlerin davranışını yansıtmaktadır. Öne taşımada son aranan kayıtlar başa toplanmakta, 21 anahtarı tek bir aramayla ikinci sıraya yükselmektedir. Konum değiştirmede 90 anahtarı dört aramaya karşın ancak ikinci sıraya gelebilmektedir. Sayaç yönteminde sıra doğrudan erişim sayısını izlemekte ve 90, 89, 21 biçiminde oluşmaktadır. Kodun elle yapılan izlemeyle karşılaştırılması, algoritmanın doğru anlaşıldığını sınamanın pratik bir yoludur.